Prompt Engineering Theory

Purpose: Comprehensive reference for prompt engineering theory, covering temperature control, accuracy optimization, hallucination mitigation, and production patterns.

Status: Authoritative **Last updated:** 2026-01-22 **Scope:** ML engineering with LLMs (focus: production systems, anti-hallucination, deterministic workflows)

Table of Contents

Core Theory

1. Temperature Theory
2. Accuracy Stack
3. Hallucination Theory
4. Tokenization Fundamentals

Practical Engineering

5. Prompt Engineering Patterns
6. Production Workflows
7. Temperature Control Strategies
8. Anti-Hallucination Protocols

Implementation

9. Prompt Frameworks
10. Model Selection
11. Tools & Platforms
12. Quick Reference

---

1) Temperature Theory

1.1 What Temperature Means (Mathematical Foundation)

LLMs generate text token by token. At each step, the model produces a vector of **logits** (unnormalized scores) over the next token vocabulary. These logits are turned into a probability distribution using softmax.

Temperature rescales logits before softmax:

$$ p_i(T) = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)} $$

Where:

- (z_i) = logit for token *i*
- (T) = temperature

Effects:

- **Low temperature (T < 1)** → probabilities become **more peaked** (more deterministic)
- **High temperature (T > 1)** → probabilities become **flatter** (more diverse)

Extreme cases:

- **T → 0:** nearly greedy decoding (argmax)
- **T → ∞:** close to uniform randomness

> **Engineering mental model:** Temperature is an **exploration strength / entropy control** knob, not kurtosis or regression sigma.

1.2 Why Temperature Changes "Creativity" and Correctness

- **Lower T** concentrates probability mass on the most likely continuations → more stable and repeatable answers
- **Higher T** increases probability of lower-ranked tokens → more variety, but also more chance of wrong branches

This is exactly why "high temperature = more hallucinations" is often observed in factual/code tasks.

1.3 What Temperature Does and Doesn't Do

Temperature DOES:

- Change how "adventurous" sampling is
- Change output diversity and stability
- Affect exploration vs exploitation

Temperature DOES NOT:

- Magically grant truthfulness
- Prevent hallucinations on its own
- Low temperature can still hallucinate — it may just hallucinate **consistently**

Root causes of hallucination:

- Missing context
- Weak grounding
- Overconfident continuation behavior
- Ambiguous prompts
- NOT just randomness

> **Practical consequence:** Temperature alone helps, but **constraints + verification loops** matter more.

1.4 Industry-Standard Temperature Values

Table 2-3 (Industry Best Practices):

Task Type	Temperature	Rationale

Task Type	Temperature	Rationale
Bug fixing / code review / data analysis	0.0–0.2	Deterministic, precise solutions
Code generation (production)	0.2–0.3	Accurate, adheres to conventions
Architecture brainstorming	0.6–0.9	Exploration, diverse ideas
Creative writing / wild ideation	~1.0+	Maximum diversity

Why these values?

Deterministic tasks (0.0–0.2):

- Narrow "correct" solution set
- Randomness is harmful (invented details, unjustified refactors)
- Push to **exploit mode**

Creative tasks (0.6–1.0):

- Diversity helps explore solution space
- Want non-obvious candidate ideas
- Then **converge** with low-temperature decision pass

1.5 Temperature vs Other Sampling Parameters

Anthropic Official Guidance:

- Default temperature: 1.0 (if not specified)
- Range: 0.0–1.0
- Use temperature closer to 0.0 for analytical/multiple choice
- Use temperature closer to 1.0 for creative/generative tasks
- **Critical:** Even at 0.0, results not fully deterministic
- **Best practice:** Adjust temperature OR top_p, but NOT both

Parameter interactions:

Parameter	Effect	When to Use
temperature	Smoothly reshapes probabilities	Primary control knob
top_p	Truncates distribution (hard gate)	Block low-prob nonsense
top_k	Limits token count	Strict vocabulary control
repeat_penalty	Penalizes repetition	Avoid loops

> **Engineering rule:** Temperature is primary. Top_p is secondary filter. Never tune both simultaneously.

1.6 Why T=0 Doesn't Guarantee Accuracy

Low temperature makes the model confidently consistent.

If the model's top continuation is wrong, it will output wrong text *more deterministically*.

Key insight:

- Low T reduces **variance**
- Low T does NOT guarantee **truth**

Therefore: Temperature control must be combined with grounding, verification, and constraints.

---

2) Accuracy Stack

Source: accuracy-theory.md + production engineering practices

2.1 What "Accuracy" Means for LLMs

Accuracy is not one thing. In practice, you care about different "accuracies":

1. **Factual accuracy:** Are claims correct (dates, features, laws, definitions)?
2. **Task accuracy / functional correctness:** Does code compile, pass tests, solve the stated bug?
3. **Reasoning accuracy:** Are steps consistent and logically valid?
4. **Groundedness (anti-hallucination):** Does model stick to provided information or invent?

Critical insight: Decoding knobs (temperature/top_p) mostly affect #4 and a bit of #2–3, but barely affect #1 unless the facts are already in-context.

> **Core principle:** Most "accuracy failures" are **missing information**, not randomness.

2.2 The Real Accuracy Stack (Ranked by Impact)

Rank #1 — Grounding / Context

The biggest lever is: what evidence the model sees.

Accuracy increases dramatically when:

- You provide relevant excerpts
- You provide code + logs + stack traces
- You provide constraints + expected behavior
- You allow tools (web / execution)

Engineering translation: LLMs are not databases. If it's not in context, it might guess.

Practical moves:

- Paste relevant files, not summaries
- Paste failing test output
- Provide "Definition of Done"
- Provide authoritative sources when facts matter

Rank #2 — Tool Grounding (Execution / Retrieval)

Tools beat sampling.

Because tools add an external truth source:

- Web search + citations for facts
- Code execution for Python
- Unit tests for code
- Linters/type-checkers

> **Best "parameter" for accuracy:** Use a workflow that **forces verification against reality**.

Production systems use:

- RAG (Retrieval-Augmented Generation)
- Tool-use agents
- Verification pipelines

Rank #3 — Model Selection (Capability + Calibration)

Model choice dominates decoding parameters.

A strong model at "creative-ish" settings often beats a weak model at T=0.

Base model properties that matter:

- Training quality (code vs general mix)
- Instruction tuning
- Calibration (probabilities match truth)
- Robustness to ambiguous prompts
- Willingness to say "I don't know"

Engineering heuristic:

- For code: choose **coder-specialized model**
- For reasoning: choose **strong reasoning model**
- For domain content: choose model with known domain strength

---

Rank #4 — Prompt Constraints

Prompting can drastically reduce hallucination even with same temperature.

Constraints that increase accuracy:

- Strict output schema (JSON/YAML)
- "No guessing" policy
- Explicit uncertainty section
- Tests required
- Cite source lines ("quote the snippet you used")

Forces model to behave like:

- Verifier
- Reviewer
- Cautious engineer

Not: Story generator

Rank #5 — Decoding Parameters (temperature / top_p / seeds)

These matter, but they're not the king.

They control **variance**:

- How likely model takes lower-probability branches
- How diverse outputs are across runs

They do NOT insert missing truth.

2.3 Among Decoding Params: What Matters Most?

Top_p is often more important than temperature for accuracy.

Why?

- **Temperature** reshapes probabilities smoothly
- **Top_p** truncates distribution (literally forbids low-prob tokens)

So top_p acts like a **hard gate:**

- If nonsense tokens are in the tail, top_p blocks them

Best settings for accuracy in practice:

```
For correctness-critical outputs:
  temperature: 0.0–0.2
  top_p: 0.8–0.95
  top_k: 20–50 (if supported)
  repeat_penalty: 1.05–1.2 (avoid loops)
```

But: Don't over-tune. The biggest effect is just:

- Low T
- Not-too-high top_p

2.4 The Paramount Accuracy Parameter

> **The #1 parameter for accuracy is NOT temperature. It is the amount and quality of grounding information in-context.**

Context quality includes:

- Relevant evidence
- Exact logs
- Exact code
- Constraints
- Definitions

If the model has truth in context window, it can copy/transform accurately. If it doesn't, it will guess.

---

3) Hallucination Theory

Source: Fano's Inequality + prompt-foundations-references.md

3.1 Fano's Inequality Applied to LLMs

Mathematical foundation:

$$ H(X|Y) \geq \frac{H(P_e) + P_e \log(M-1)}{1} $$

Where:

- (X) = true answer
- (Y) = LLM output
- (P_e) = probability of error (hallucination)
- (M) = size of output space
- (H(X|Y)) = conditional entropy (ambiguity given output)

Key insight: Hallucination becomes **inevitable** when prompts are ambiguous (high (H(X|Y))).

Three levers to reduce hallucinations:

1. **Reduce output space (M)**
    - Use structured outputs (JSON/XML)
    - Constrain vocabulary
    - Specify exact format
2. **Reduce ambiguity (H)**
    - Clear, specific prompts
    - Provide context
    - Eliminate multiple valid interpretations
3. **Design for uncertainty floor**
    - Acknowledge hallucinations are inevitable
    - Build verification workflows
    - Require uncertainty quantification

3.2 Top Hallucination Reducers (Evidence-Based)

Ranked by effectiveness:

1. **RAG (Retrieval-Augmented Generation)**
   - Gold standard for factual accuracy
   - Grounds model in external knowledge
   - Reduces (M) and (H) simultaneously
2. **CoVe (Chain-of-Verification)**
   - Generate answer
   - Generate verification questions
   - Answer verification questions
   - Produce final verified answer
3. **Step-Back Prompting**
   - Ask model to step back and consider principles
   - Reduces rushing to wrong conclusions
   - Improves reasoning accuracy
4. **Structured Outputs (XML/JSON)**
   - Reduces output space (M)
   - Forces format compliance
   - Easier to validate
5. **Cognitive Verifier**
   - Reduce ambiguity via clarifying questions
   - Model asks for missing information
   - Human provides clarification

3.3 Anti-Hallucination Protocols

Protocol 1: Verification Loop

```
Audit your previous answer.

1) Extract all factual claims.
2) Label each claim: Certain / Likely / Speculative.
3) Remove all Speculative claims unless justified from first principles.
4) Rewrite the answer using only Certain + Likely claims.
5) Provide a short "What to verify externally" list (max 5).
```

Effect: Behaves like "temperature reduction" by forcing:

- Explicit uncertainty
- Constrained output
- Fewer creative branches

Protocol 2: Deterministic Engineering Mode

```
Switch to deterministic engineering mode.

Rules:
- No guessing. If unsure, say "unknown / needs verification".
- Output only:
  (1) best solution
  (2) minimal change
  (3) tests to validate
  (4) risks / failure modes
- Do NOT add extra suggestions.
```

Effect: Shrinks model's search space without API temperature control.

Protocol 3: Citation-Required Mode

```
Every factual claim must include:
- Source (document, line number, URL)
- Exact quote (if from provided context)
- Confidence level (High / Medium / Low)

If source unavailable, mark as [UNVERIFIED].
```

Effect: Forces grounding, makes hallucinations explicit.

3.4 When Hallucinations Are Most Likely

High-risk scenarios:

1. **Missing context**
   - Model doesn't have information
   - Tries to be helpful by guessing
2. **Ambiguous prompts**
   - Multiple valid interpretations
   - Model picks one without checking
3. **Overconfident continuation**
   - Model continues plausible-sounding pattern
   - Pattern is wrong but coherent
4. **Edge cases / rare domains**
   - Training data sparse

    - Model interpolates incorrectly

5. **Conflicting information**
    - Context contains contradictions
    - Model resolves without flagging

Mitigation for each:

Risk	Mitigation
Missing context	Provide explicit sources, use RAG
Ambiguous prompts	Use structured formats, clarifying questions
Overconfident continuation	Force uncertainty labeling, verification loops
Edge cases	Acknowledge limits, require "unknown" answers
Conflicting information	Explicit conflict resolution, cite sources

---

4) Tokenization Fundamentals

Source: prompt-foundations-references.md (Part 1)

4.1 What Tokenization Is

Definition: Process of breaking text into discrete units (tokens) that models can process.

Key insight: Different models use different tokenizers, leading to varying token counts for identical text.

Why it matters:

- Token count affects cost (API pricing)
- Token count affects context window usage
- Tokenizer choice impacts efficiency

4.2 Tokenizer Types

Common approaches:

1. **Character-level**
    - Smallest units
    - Large vocabulary, simple implementation
    - Rarely used in modern LLMs

2. **Word-level**
    - Natural language units
    - Handles OOV (out-of-vocabulary) poorly
    - Limited use

3. **Subword tokenization (BPE, WordPiece, SentencePiece)**

- **Industry standard**
- Balances vocabulary size and coverage
- Handles rare words via decomposition

Example (BPE):

```
"tokenization" → ["token", "ization"]
"anthropic" → ["anth", "rop", "ic"]
```

4.3 Embeddings Overview

Two main types:

1. Token Embeddings (Contextualized)

- Used internally by LLMs
- Each token has embedding that changes with context
- Example: "bank" in "river bank" vs "savings bank"

2. Sentence/Document Embeddings

- Fixed-size representation of entire text
- Used for similarity, retrieval, clustering
- Example: Semantic search in RAG systems

Model choices:

- **Word2Vec:** Still useful for recommendations, similarity (legacy but fast)
- **BERT embeddings:** Contextualized, good for classification
- **Sentence transformers:** Optimized for semantic similarity

4.4 Impact on Prompt Engineering

Practical considerations:

1. **Token counting**

   ```
   # Different tokenizers, different counts
   text = "Hello, world!"
   gpt4_tokens = len(tiktoken.encode(text))  # e.g., 4 tokens
   claude_tokens = len(anthropic_tokenizer.encode(text))  # e.g., 3 tokens
   ```

2. **Cost optimization**

   - Shorter prompts = lower cost
   - But: completeness matters more than brevity
   - Don't sacrifice grounding to save tokens

3. **Context window management**
    - Track token usage
    - Prioritize most relevant context
    - Use summarization for long documents

---

5) Prompt Engineering Patterns

Source: prompt-foundations-references.md (Part 3) + vibe-coding-guide.md

5.1 Fundamental Techniques

Zero-shot:

- Just describe the task clearly
- No examples needed
- Good for: Simple, well-defined tasks

One-shot:

- Provide **one example**
- Shows format and style
- Good for: Format-sensitive tasks

Few-shot:

- Provide **multiple examples**
- Consistency skyrockets
- Good for: Complex tasks, specific styles

Chain-of-Thought (CoT):

- Ask model to reason step-by-step
- Improves logical accuracy
- Must be guided (not "think freely")

ReAct (Reason + Act):

- Reason about next action
- Execute action with tools
- Observe result, repeat
- Good for: Agent workflows

Self-Critique:

- Ask AI to review and improve its own answer
- Catches obvious errors
- Reduces hallucinations

5.2 Production Prompt Patterns

Pattern 1: Design First, Code Later

Bad:

```
Write the code.
```

Good:

```
First propose a design.
List trade-offs.
Then implement the simplest solution.
Add tests.
```

Pattern 2: Code Review

```
Review this code for:
- Security risks
- Performance issues
- Dead branches
- Missing tests

Explain why each issue matters.
```

Pattern 3: Debugging

Always provide:

```
Observed behavior: [what happens]
Expected behavior: [what should happen]
Error logs: [full stack trace]
What I tried: [previous attempts]
```

Pattern 4: Translation

```
Convert this function from X → Y.
Preserve behavior exactly.
Explain key differences in approach.
```

Pattern 5: Guardrails

Require:

- Comments explaining WHY (not just what)
- Minimal dependencies
- Tests (unit + integration)
- Security checks

5.3 Structured Output Patterns

XML (Claude-optimized):

```
<task>
  <objective>Refactor data loader</objective>
  <constraints>
    <constraint>No breaking changes</constraint>
    <constraint>Add type hints</constraint>
  </constraints>
  <deliverables>
    <file>src/data/loader.py</file>
    <tests>tests/test_loader.py</tests>
  </deliverables>
</task>
```

JSON (GPT-optimized):

```
{
  "task": "Refactor data loader",
  "constraints": [
    "No breaking changes",
    "Add type hints"
  ],
  "deliverables": {
    "code": "src/data/loader.py",
    "tests": "tests/test_loader.py"
  }
}
```

Benefits:

- Reduces output space (M)
- Forces schema compliance
- Easier to validate programmatically

---

6) Production Workflows

Source: vibe-coding-guide.md + ai-usage-policy-comprehensive.md

6.1 Four-Phase Workflow

Phase 1: Explore (High Temperature ~ 0.7–0.9)

```
Generate 3–5 alternative approaches.
For each, list:
- Pros
- Cons
- Risks
```

Phase 2: Decide (Low Temperature ~ 0.2)

```
Pick ONE approach from exploration.
Justify choice based on:
- Optimization priority (latency/accuracy/velocity)
- Risk profile
- Implementation complexity
```

Phase 3: Implement (Low Temperature ~ 0.2)

```
Minimal diff + tests.
No refactors unless required.
Exact file paths.
Complete code blocks (not pseudocode).
```

Phase 4: Verify (Lowest Temperature ~ 0.0–0.1)

```
Audit implementation:
1) Extract assumptions
2) List failure modes
3) Define validation approach
4) Provide rollback plan
```

6.2 Vibe Coding Principles Applied

Core mindset:

- Human: Sets goals, verifies, holds accountability
- AI: Executes, drafts, explores, iterates

Small steps:

- One task per prompt

- Max ~200 lines changed
- Frequent re-summarization

Isolate risks:

- Failure analysis upfront
- Validation strategy before code
- Git discipline (review diff before commit)

Tests before merging:

- Success criteria measurable
- Exact test commands
- No "works well" vagueness

Never fully trust:

- Human reviews all diffs
- Runs validation commands
- Reverts if unsure

6.3 Verification-First Paradigm

From "Traditional Code Review Is Dead":

Old paradigm:

- Human reviews every line
- Relies on human attention

New paradigm:

- Automated verification (tests, CI, type-checking)
- Human reviews architecture/risk
- NOT syntax/style

Quality stack:

1. Tests (unit + integration + smoke)
2. CI gates (lint, type-check, security scan)
3. Preview environments (visual validation)
4. Security scanning (dependency audit)
5. Human review (architecture, edge cases, risk)

Branch protection:

```
# .github/branch_protection.yml
require_status_checks:
  - tests_pass
  - lint_clean
  - type_check_pass
  - security_scan_clean
```

```
require_reviews: 1
dismiss_stale_reviews: true
require_code_owner_reviews: true
```

---

7) Temperature Control Strategies

Source: models-temperature-theory.md + practical engineering

7.1 UI vs API Reality

In consumer chat UIs (Claude.ai, ChatGPT):

- ✗ Cannot set temperature directly
- ✗ Cannot reliably infer exact sampling settings

In APIs / local inference (Ollama, Anthropic API):

- ✓ Can set temperature explicitly
- ✓ Full control over sampling parameters

Engineering rule:

> **UI = convenience API/local inference = knobs + reproducibility**

7.2 "Effective Temperature Control" Without Knobs

If you cannot set temperature, shrink model's search space via prompts:

Deterministic Engineering Mode

```
Switch to deterministic engineering mode.

Rules:
- No guessing. If unsure, say "unknown / needs verification".
- Output only:
  (1) best solution
  (2) minimal change
  (3) tests to validate
  (4) risks / failure modes
- Do NOT add extra suggestions.
```

Verification Loop

```
Audit your previous answer.

1) Extract all factual claims.
```

```
2) Label each claim: Certain / Likely / Speculative.
3) Remove all Speculative claims unless justified from first principles.
4) Rewrite the answer using only Certain + Likely claims.
5) Provide a short "What to verify externally" list (max 5).
```

Effect: Behaves like temperature=0.2 reduction.

7.3 Local Control (Ollama Examples)

Accuracy Mode (Debugging / Production Code):

```
ollama run qwen3-coder \
  --temperature 0.15 \
  --top-p 0.9 \
  --repeat-penalty 1.1
```

Prompt:

```
Do not guess.
If uncertain, say UNKNOWN.
Provide:
1) diagnosis (ranked)
2) minimal fix
3) tests (exact commands)
```

Verifier Mode (Post-Answer Audit):

```
ollama run qwen3-coder \
  --temperature 0.05 \
  --top-p 0.85
```

Prompt:

```
Audit your previous answer.
List every factual claim.
Mark: Certain / Likely / Speculative.
Remove Speculative.
Rewrite in final form.
```

Exploration Mode (Architecture Design):

```
ollama run qwen3-coder \
  --temperature 0.7 \
  --top-p 0.95
```

Prompt:

```
Generate 3 architectures.
Then choose 1 and justify tradeoffs.
```

Then rerun chosen design through Verifier mode.

7.4 API Configuration (Production)

Ollama with Anthropic API compatibility:

```python
import anthropic

client = anthropic.Anthropic(
    base_url='http://localhost:11434',
    api_key='ollama',  # required but ignored
)

# Temperature based on task type
TEMPERATURE_CONFIG = {
    "code_generation": 0.2,
    "code_review": 0.2,
    "bug_fixing": 0.2,
    "data_analysis": 0.2,
    "creative_brainstorming": 0.8,
}

response = client.messages.create(
    model='qwen3-coder',
    max_tokens=4096,
    temperature=TEMPERATURE_CONFIG["code_generation"],
    messages=[{"role": "user", "content": "..."}]
)
```

Critical: Never combine temperature with `top_p` or `top_k` (per Anthropic guidelines).

7.5 Mode Prompts (Reusable)

MODE_DEBUG (Anti-Hallucination):

```
You are in MODE_DEBUG.
No guessing. If uncertain, say "unknown".
```

```
Return:
1) diagnosis hypotheses (ranked)
2) minimal experiments to confirm
3) minimal fix
4) regression tests
```

MODE_CODEGEN (Production):

```
You are in MODE_CODEGEN.
Generate only minimal diff code + tests.
No refactor unless required.
Return:
A) files to change
B) patch-style code blocks
C) commands to run
D) definition of done
```

MODE_BRAINSTORM (Exploration):

```
You are in MODE_BRAINSTORM.
Generate 3 approaches with pros/cons.
Then pick the best one and commit.
Return:
- chosen approach
- step plan
- risks
```

---

8) Anti-Hallucination Protocols

Source: Integrated from accuracy-theory.md + hallucination research

8.1 The Accuracy Protocol (Step-by-Step)

Step A — Ground

Provide:

- Objective
- Constraints
- Artifacts (code/logs/data excerpt)

Step B — Force Structure

Require:

- Assumptions
- Uncertainties

- Tests

Step C — Tool Verify

Use:

- Unit tests / Python execution
- Citations for facts
- Web search for current info

Step D — Audit Pass

After first response:

```
Extract all claims.
Label certainty.
Rewrite without speculative claims.
```

8.2 Concrete Configs for Maximum Accuracy

For Ollama (ML/CV Production):

```
Task: Bug fixing / code review
Config:
  temperature: 0.1
  top_p: 0.9
  repeat_penalty: 1.1

Prompt pattern:
  "Do not guess. If uncertain, say UNKNOWN.
   Provide:
   1) diagnosis (ranked)
   2) minimal fix
   3) tests (exact commands)"
```

```
Task: Verification / audit
Config:
  temperature: 0.05
  top_p: 0.85

Prompt pattern:
  "Audit your previous answer.
   List every factual claim.
   Mark: Certain / Likely / Speculative.
   Remove Speculative.
   Rewrite in final form."
```

8.3 When Temperature Is Paramount

Temperature becomes the main lever ONLY when:

1. Model already has adequate context
2. You care mainly about diversity or creativity
3. Tasks are underdetermined

Examples:

- Brainstorming model architectures
- Generating alternative refactor designs
- Writing docs in different styles
- Ideating experiments

For factual correctness and debugging: Temperature is secondary to grounding.

8.4 Red Flags for Hallucination

Immediate rejection criteria:

- ✕ Invented API functions that don't exist
- ✕ Plausible-sounding but wrong package names
- ✕ Deleted tests instead of fixing them
- ✕ Overly complex solutions for simple problems
- ✕ Missing edge cases
- ✕ Ignored constraints from prompt
- ✕ No uncertainty acknowledgment

Validation checklist:

- ☐ All APIs exist (check documentation)
- ☐ All packages exist (pip search, PyPI)
- ☐ Tests pass (actually run them)
- ☐ Constraints satisfied (review prompt)
- ☐ Edge cases considered (failure analysis)
- ☐ Uncertainty acknowledged (where applicable)

---

9) Prompt Frameworks

Source: prompt-foundations-references.md (Section 4.10)

9.1 COSTAR Framework

Components:

- **C**ontext: Background information
- **O**bjective: Clear task definition
- **S**tyle: Desired output format/tone
- **T**one: Communication style

- **A**udience: Who will use this
- **R**esponse: Expected output format

Example:

```
Context: ML pipeline for defect detection (500 images, 4096×4096)
Objective: Design data splitting strategy to prevent leakage
Style: Technical documentation
Tone: Direct, rigorous
Audience: Senior ML engineers
Response: Architecture diagram + validation approach + exact commands
```

9.2 CRISPE Framework

Components:

- **C**apacity and Role: Who the AI is
- **I**nsight: Background context
- **S**tatement: Task description
- **P**ersonality: Communication style
- **E**xperiment: Output format

Example:

```
Capacity: Senior ML engineer specializing in CV
Insight: Working on production defect detection (tight latency budget)
Statement: Optimize inference pipeline to meet <30ms p99 target
Personality: Direct, practical, no theory dumps
Experiment: Return benchmark results table + profiling data + optimization plan
```

9.3 RTF Framework (Role, Task, Format)

Simplest framework:

```
Role: You are a [specific role]
Task: [Exact task description]
Format: [Output structure]
```

Example:

```
Role: You are my senior ML/CV engineering partner
Task: Design dataset splitting strategy for 5k PCB images with 1:100 defect ratio
Format: Return architecture + failure analysis + validation commands
```

9.4 Which Framework to Use

Decision matrix:

Framework	Best For	Complexity
RTF	Quick tasks, simple prompts	Low
COSTAR	Comprehensive tasks, multiple stakeholders	Medium
CRISPE	Personality-sensitive tasks, creative work	Medium-High

For ML/CV production work: RTF is usually sufficient. Use COSTAR for complex multi-stakeholder projects.

---

10) Model Selection

Source: prompt-foundations-references.md + practical experience

10.1 Model Choice vs Temperature

Picking a model is NOT marketing — it changes baseline behavior:

Model properties that differ:

- Training mix (code vs general)
- Instruction tuning quality
- RLHF/RLAIF shaping
- Calibration quality (probabilities match truth)
- Tool grounding policies
- Willingness to say "I don't know"

However: Model choice ≠ temperature control

You still need:

- Low-temp decoding for correctness tasks
- Verification loops
- Tests / execution when possible

Rule:

> Use code-specialized models for code tasks Use reasoning-oriented models for system design But **use protocol** to keep output deterministic

10.2 Model Recommendations by Task

For code generation/review:

- **Anthropic:** Claude Sonnet 4.5 (code-specialized)
- **OpenAI:** GPT-4 Turbo
- **Local:** Qwen3-Coder, DeepSeek-Coder

For reasoning/system design:

- **Anthropic:** Claude Opus 4.5
- **OpenAI:** o1, o3
- **Local:** Qwen3, LLaMA3

For CV/ML domain:

- Prefer models with strong STEM training
- Check for math/code performance
- Test on domain-specific benchmarks

10.3 Model-Specific Strengths

Claude (Anthropic):

- ✅ Best with XML + structure
- ✅ Excellent context management (200k tokens)
- ✅ Strong refusal calibration (says "I don't know")
- ✅ Good for long documents, complex prompts

GPT (OpenAI):

- ✅ Agentic strengths (function calling)
- ✅ Clean markdown delimiters
- ✅ Strong tool use
- ✅ Good for multi-step workflows

Local models (Ollama):

- ✅ Privacy (no data leaves machine)
- ✅ Full control (temperature, sampling)
- ✅ No API costs
- ✅ Air-gapped deployment

---

11) Tools and Platforms

Source: mcp-references.md + prompt-foundations-references.md

11.1 MCP (Model Context Protocol)

Official registries:

1. **Official MCP Registry**
    - URL: registry.modelcontextprotocol.io
    - API access for searching servers
    - Status: Preview (v0.1)
2. **MCP.so** (Community Marketplace)

    - 17,262+ MCP servers
    - Community-driven

3. **PulseMCP** (Auto-Updated)
    - 7,530+ servers
    - Daily updates

For robotics/CV focus, search for:

- Arduino/robotics servers
- ROS integration
- Computer vision/perception
- Sensor fusion
- Real-time control

Programmatic access:

```
# Official Registry API
curl "https://registry.modelcontextprotocol.io/v0/servers?limit=10"
curl "https://registry.modelcontextprotocol.io/v0/servers?search=robotics"
```

11.2 Prompt Engineering Platforms

LangSmith (LangChain):

- Prompt versioning
- A/B testing
- Evaluation datasets
- Production monitoring

PromptLayer:

- Prompt tracking
- Version control
- Analytics
- Team collaboration

Weights & Biases Prompts:

- Experiment tracking
- Prompt comparison
- Visualization
- Integration with W&B ecosystem

11.3 Evaluation Tools

For factual accuracy:

- RAGAS (RAG assessment)

- TruLens (LLM evaluation)
- DeepEval (production testing)

For code quality:

- CodeBERT (code similarity)
- Unit test execution
- Linting (ruff, pylint)
- Type checking (mypy)

---

12) Quick Reference

12.1 Temperature Cheat Sheet

Accuracy / correctness tasks:

```
Target: Deterministic output
Temperature: 0.0–0.2
Top_p: 0.8–0.9

Tactics:
  - Strict schema
  - Minimal diffs
  - Force test plan
  - Audit factual claims
```

Ideation tasks:

```
Target: Diversity
Temperature: 0.7–0.9
Top_p: 0.95

Tactics:
  - Generate multiple approaches
  - Then converge with low-temp decision pass
```

12.2 Hallucination Mitigation Checklist

- ☐ Provide relevant context (code, logs, docs)
- ☐ Use structured output (XML/JSON)
- ☐ Require uncertainty labeling (Certain/Likely/Speculative)
- ☐ Force citation of sources
- ☐ Run verification loop (audit pass)
- ☐ Use tools for ground truth (execution, web search)
- ☐ Low temperature (0.0–0.2)
- ☐ Explicit "no guessing" policy

12.3 Production Workflow Template

```
Phase 1 - Explore (T=0.7):
  - Generate 3-5 approaches
  - Pros/cons/risks for each

Phase 2 - Decide (T=0.2):
  - Pick ONE approach
  - Justify based on priorities

Phase 3 - Implement (T=0.2):
  - Minimal diff + tests
  - No unnecessary refactors

Phase 4 - Verify (T=0.05):
  - Extract assumptions
  - List failure modes
  - Define validation
  - Rollback plan
```

12.4 Prompt Quality Checklist

Before sending:

- ☐ Task clearly defined (measurable outcome)
- ☐ Context complete (code/logs/constraints)
- ☐ Output format specified (structure)
- ☐ Examples provided (few-shot if complex)
- ☐ Constraints explicit (what NOT to do)
- ☐ Success criteria measurable

Before accepting output:

- ☐ Assumptions reasonable
- ☐ No hallucinated APIs/packages
- ☐ Tests defined and passing
- ☐ Failure modes addressed
- ☐ Uncertainty acknowledged
- ☐ Can rollback if wrong

12.5 Common Anti-Patterns

✖ Avoid:

- Vague prompts ("make it better")
- No context ("fix this")
- No format ("give me ideas")
- Accepting first output without verification
- Combining temperature + top_p tuning

- Over-reliance on sampling without grounding

✅ **Do:**

- Specific, measurable tasks
- Complete context (evidence-based)
- Structured output formats
- Verification loops
- Temperature OR top_p (not both)
- Grounding + verification + constraints

12.6 Emergency Debugging

If AI output is wrong:

1. **Ask for self-critique**

   ```
   "Explain your assumptions.
    List 3 ways this could fail.
    What might be hallucinated?"
   ```

2. **Add missing context**

   - Actual logs (not summaries)
   - Actual error messages (full stack trace)
   - Actual constraints (hardware, budget)

3. **Break into smaller tasks**

   - One file at a time
   - One function at a time
   - Max ~200 lines changed

4. **Reduce randomness**

   - Lower temperature (0.7 → 0.1)
   - Stricter constraints
   - Explicit examples

5. **Verify externally**

   - Check package existence (pip search)
   - Check API existence (official docs)
   - Run tests (actually execute)

Summary

Core principles:

1. **Temperature controls variance, NOT truth**
    - Low T = deterministic
    - High T = diverse
    - Neither guarantees correctness
2. **Accuracy = Grounding + Verification + Constraints**
    - Context quality (#1 priority)
    - Tools for ground truth (#2)
    - Model selection (#3)
    - Prompt constraints (#4)
    - Temperature (#5)
3. **Hallucinations are inevitable (Fano's Inequality)**
    - Reduce output space (M)
    - Reduce ambiguity (H)
    - Design for uncertainty floor
4. **Production = Verification-First**
    - Tests before merge
    - CI gates mandatory
    - Human reviews architecture
    - Automated reviews syntax
5. **Workflows > Parameters**
    - Explore → Decide → Implement → Verify
    - Tools beat sampling
    - Verification loops essential

For ML/CV production work:

- Default temperature: 0.2
- Mandatory: Verification protocol
- Critical: Grounding (code, logs, constraints)
- Non-negotiable: Tests + validation

Sources

- models-temperature-theory.md (temperature theory)
- accuracy-theory.md (accuracy stack)
- prompt-foundations-references.md (Fano's Inequality, patterns, frameworks)
- vibe-coding-guide.md (production workflows)
- ai-usage-policy-comprehensive.md (verification-first paradigm)
- mcp-references.md (MCP registries)
- Anthropic Documentation (official guidance)
- Industry best practices (Mobileye/Waymo-class systems)